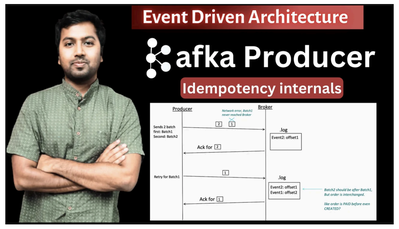
15: Transaction in kafka (Producer)

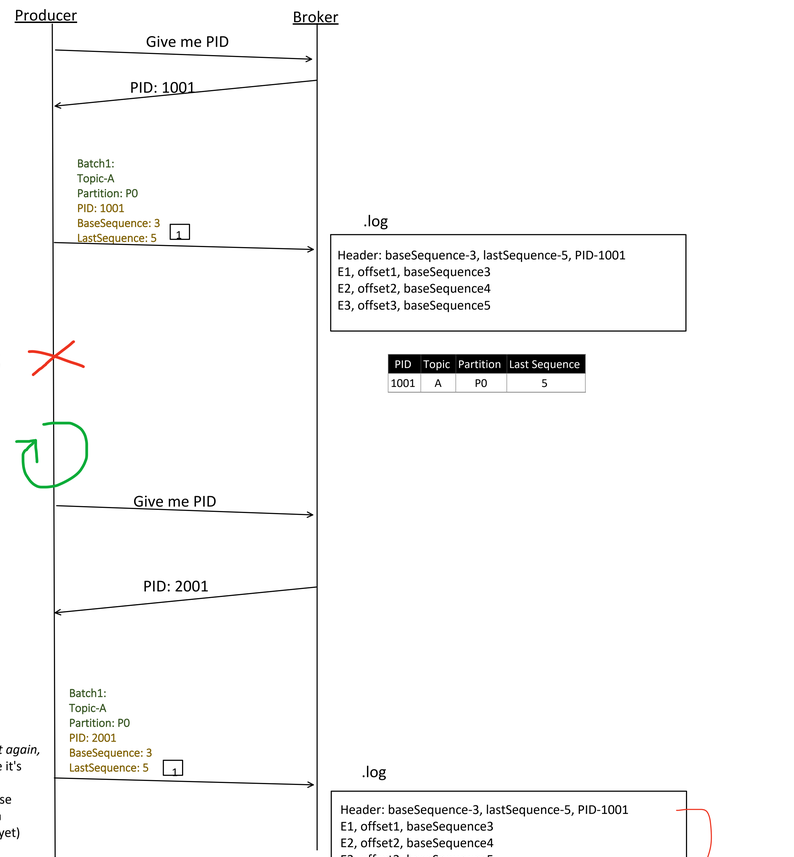
Saturday, 28 March 2026 10:55 AM

Lets understand the need for Transaction in Kafka:

1. Problem with Idempotency - Producer side Problem



So, when Producer restart, it will send request again "Give me Unique ID" and since Controller (in case of Idempotency) do not store any producer and PID mapping info, so it will always generate new PID and revert back.



PID	Topic	Partition	Last Sequence
1001	A	P0	5

PID	Topic	Partition	Last Sequence
1001	A	P0	5

2001	A	P0	5

2. Partial Writes - Producer side problem

```
//success
kafkaTemplate.send("order-events", order);

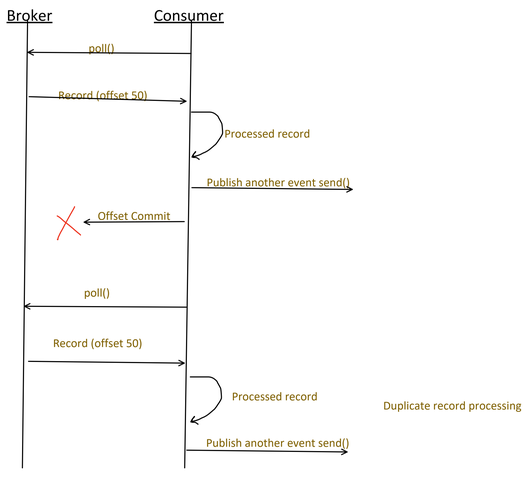
//success
kafkaTemplate.send("payment-events", payment);

//Fails(broker down or network issue)
kafkaTemplate.send("inventory-events", stock);
```

Outcome:
"order-events" and "payment-events" have records,
But "inventory-events" does NOT.
Consumers see partial data which could led to INCONSISTENT STATE

Payment made but stock not RESERVED.

3. Duplicate Processing (Read-process-Write) - Consumer side problem



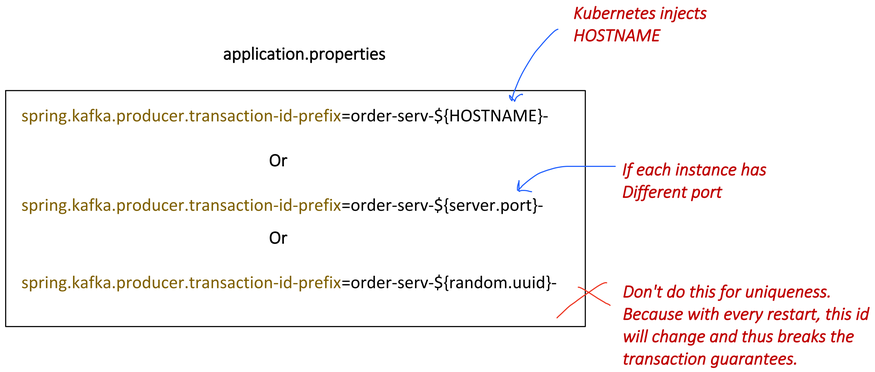
SOLUTION: TRANSACTION in Kafka

Step1: Transaction Id

- Each producer assign themselves unique transaction id.
- If Producer application has multiple instances, then each instance should have different unique id.
- Unique id should be stable, means if application restarts this id should not be changed.

application.properties

spring.kafka.producer.transaction-id-prefix=order-serv-${HOSTNAME}-



Now question might have come, why each instance of Producer application, need to have unique id?

This is needed to Avoid : ZOMBIE FENCING

We will get to know about this "Zombie Fencing" soon as we proceed further in this topic, because as this point it will be too early to cover this and it will make things complicated.

Step2: Find the Transaction Coordinator

-> __transaction_state topic holds the transaction related information.

-> Considered it as the brain of transactions.

-> This topic __transaction_state has 50 partitions. These partitions are divided among different brokers.

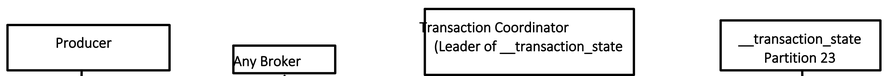
Topic: __transaction_state
Partition:
P0 - Leader Broker1
P1 - Leader Broker2
P2 - Leader Broker3
.
.
.
P49 - Leader Broker1

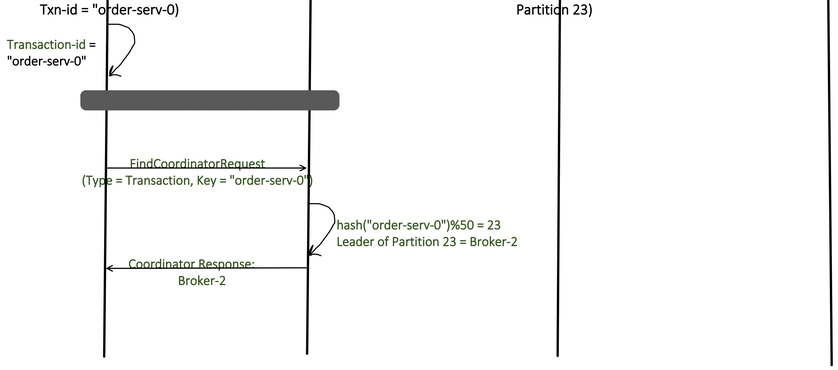
-> Particular Producer transactions goes to specific partition of __transaction_state topic only.

For example:
Producer Transaction-id = "order-serv-0"
Partition calculation = hash("order-serv-0") % 50 = Partition23

Leader of Partition23 of topic __transaction_state = Broker2

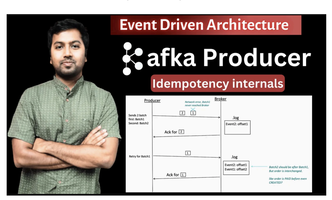
So Broker2 is considered as "Transaction Coordinator" for Producer with Transaction-id = "order-serv-0"





Step3: Get Producer ID and Version

- When we use Transaction in Kafka, internally it also enables Idempotency too.
- And idempotency need PID (Producer ID).

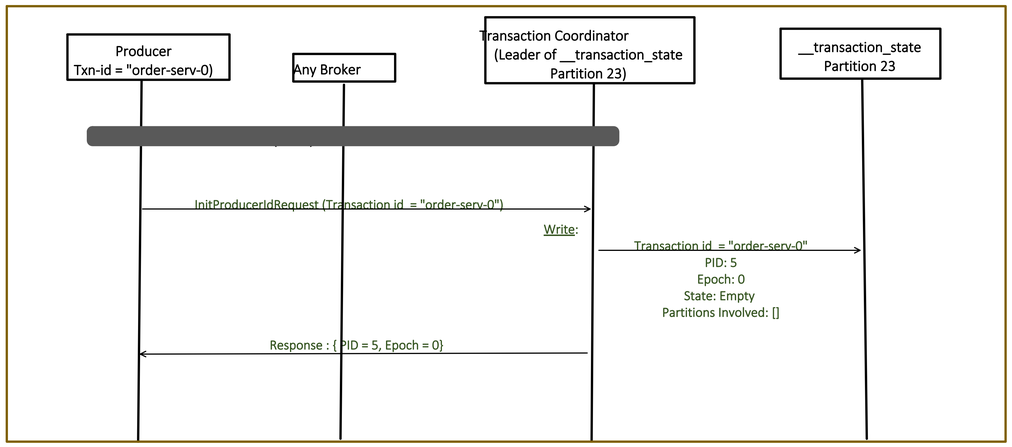


Any doubt regarding, why idempotency is used, kindly check this video.

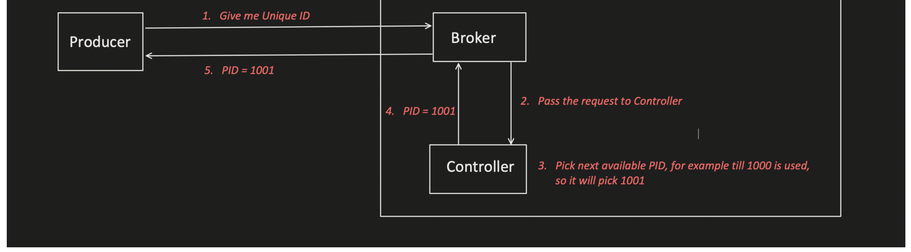
In short: Idempotent producer ensures no duplicates(even with retries) and ordering per partition (via sequence no).

And transaction need idempotency because it want retries to be safe (to avoid duplicates).

Get the Producer ID and Version



From Idempotency topic, if we recall:



So, when Producer restart, it will send request again "Give me Unique ID" and since Controller (in case of Idempotency) do not store any producer and PID mapping info, so it will always generate new PID and revert back.

This issue is resolved in Transaction, as *__transaction_state* topic, hold the mapping.

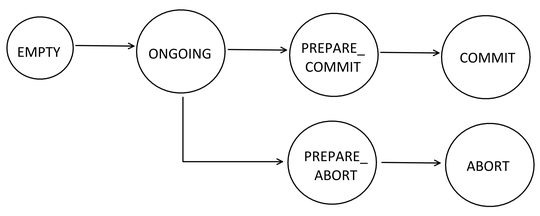
__transaction_state:

```
Transaction Id -> {
                    PID,
                    Epoch,
                    State,
                    Partitions involved,
                    Timeout,
                  }
```

PID : this is unique Producer ID

Epoch: consider it as Producer Version number (0 or 1 or 2 likewise)

State: Transaction state (EMPTY, ONGOING, PREPARE_COMMIT, PREPARE_ABORT, COMMIT, ABORT)



Partitions Involved: in partition ongoing transaction, what all partitions are involved. Like [{Topic-A, Parition2}, {Topic-B, Partition1}]

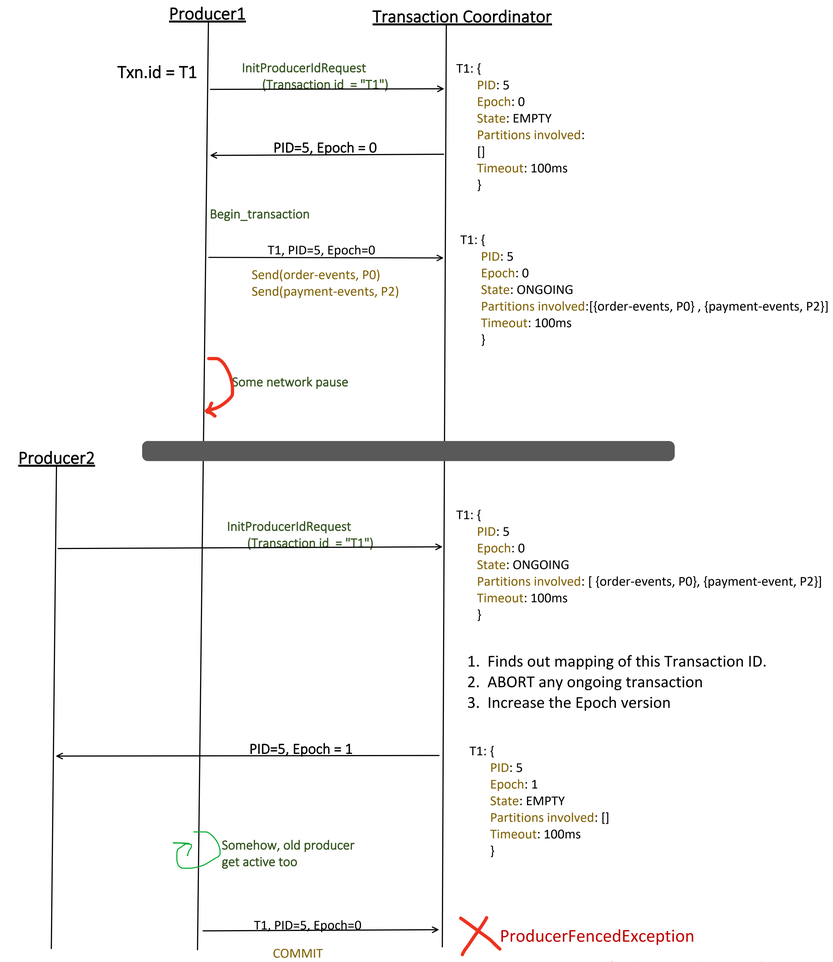
Timeout: Maximum time this Transaction can remain in ONGOING state, after that transaction coordinator will ABORT it.

Advantage of keeping information like this:

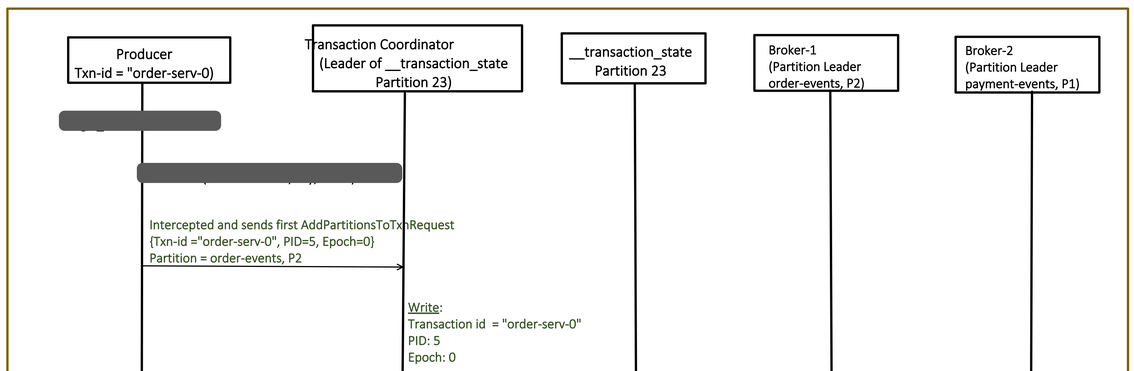
1. Producer with same Transaction Id, will always get the same PID (even when they restart). So problem with Idempotency (duplicate issue when producer restarts, as they get new PID) is now resolved with this.

2. PID + Epoch helps in ZOMBIE FENCING.

Lets understand what is Zombie Fencing.



Step4: Begin Transaction and Send Events



State: ONGOING
Partitions Involved: [{order-events, P2}]

OK

Send "order-events", key, order)
Each Event in header now also has: PID and Epoch

ACK

Intercepted and sends first AddPartitionsToTxnRequest
{Txn-id ="order-serv-0", PID=5, Epoch=0}
Partition = payment-events, P1

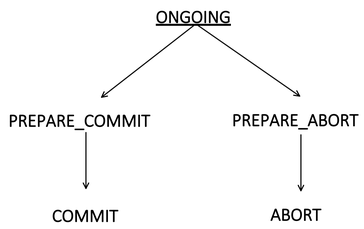
Write:
Transaction id = "order-serv-0"
PID: 5
Epoch: 0
State: ONGOING
Partitions Involved:
[{order-events, P2}, {payment-events, p1}]

OK

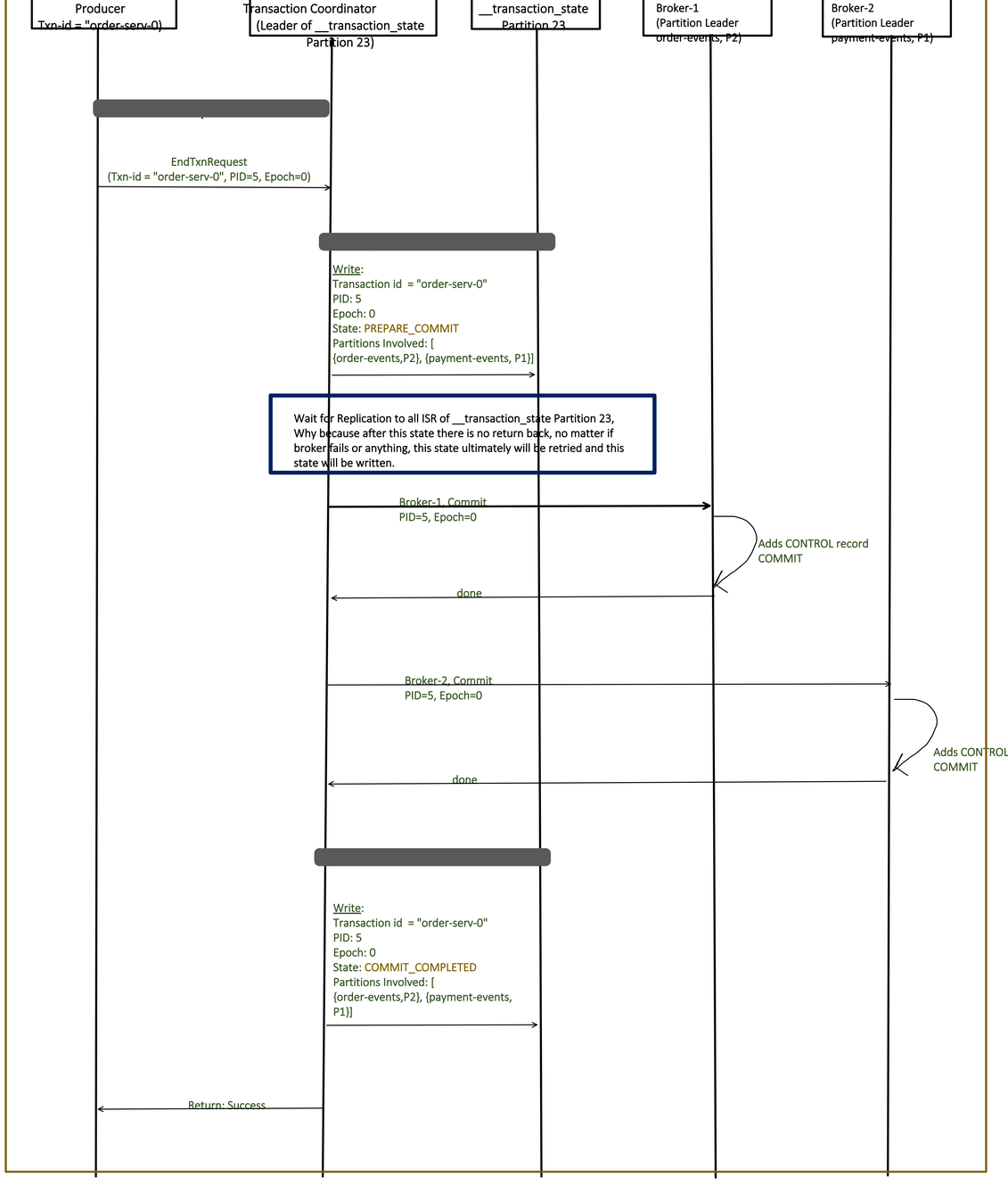
Send "payment-events", key, payment)
Each Event in header now also has: PID and Epoch

ACK

Step5: Two Phase Protocol



1st: ONGOING to PREPARE_COMMIT phase



2nd: ONGOING to PREPARE_ABORT phase

Same as COMMIT, no difference, only thing is instead of:

first, PREATE_ABORT is made and when all partitions involved in transactions successfully writes ABORT control record.
then
COMPLETED_ABORT state is updated.

And when another new transaction Started: The whole process repeats form:

Step4: Begin Transaction and Send Events

Implementation:

application.properties

```
spring.kafka.producer.transaction-id-prefix=order-serv-
#60sectimeoutfortransaction
spring.kafka.producer.properties.transaction.timeout.ms=60000
```

Just a Controller method:

```
@PostMapping("/txn-commit")
public ResponseEntity<String> txnCommit(@RequestBody Order order) {

   Payment payment = new Payment("PAY-" + order.getOrderId(), order.getCustomerId(), order.getTotalAmount(),"PAID");
   String result = orderProducerService.sendTransactionalCommit(order, payment);
   return ResponseEntity.ok(result);
}
```

OrderProducerService.java

```
@Autowired
KafkaTemplate<String, Object> kafkaTemplate;


public String sendTransactionalCommit(Order order, Payment payment) {

    return kafkaTemplate.executeInTransaction(ops -> {

                    // Send 1: order-events
                    SendResult<String, Object> orderResult = ops.send("order-events", order.getOrderId(), order).join();


                   // Send 2: payment-events (same transaction)
                   SendResult<String, Object> paymentResult = ops.send("payment-events", payment.getPaymentId(), payment).join();

                   return "COMMITTED";
          });
}
```

All send() which need to be atomic, must happens within the same transaction i.e "executeInTransaction"

Just a sample of "executeInTransaction" method

```
public <T> T executeInTransaction(OperationsCallback<K, V, T> callback) {

producer = producerFactory.createProducer(this.transactionIdPrefix);

producer.beginTransaction();

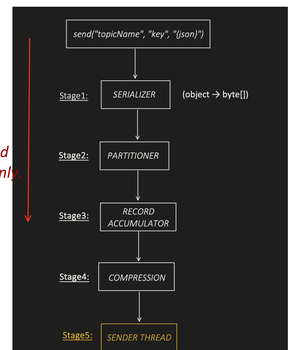
 try {
T result = callback.doInOperations(this);

producer.commitTransaction();
return result;
}
catch (Exception ex) {
producer.abortTransaction();
throw ex;
}
}
```

This join() is important too, because we know that send() is async.

Recall from Producer setup topic:

Application thread work is till here only. And free to take other requests.



With join() application thread will wait for the ACK, if there is any failure, it will abort the transaction.

Header has PID, epoch, isTransactional info

order-events logs:

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875887784 size: 243 magic: 2 compresscodec: snappy crc: 1578994549 isvalid: true
| offset: 0 CreateTime: 1774875887784 keySize: 4 valueSize: 113 sequence: 0 headerKeys: [__TypeId__] key: O-33 payload: {"orderId":"O-33","customerId":"CUST-2","productId"
:"PROD-3","quantity":1,"totalAmount":99.99,"status":"CREATED"}
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH
orizonMs: OptionalLong.empty position: 243 CreateTime: 1774875888150 size: 78 magic: 2 compresscodec: none crc: 2450373141 isvalid: true
| offset: 1 CreateTime: 1774875888150 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [] endTxnMarker: COMMIT coordinatorEpoch: 0
```

Control Record, Whether COMMIT or ABORT.

payment-events logs:

```
baseOffset: 0 lastOffset: 0 count: 1 baseSequence: 0 lastSequence: 0 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: false deleteHo
rizonMs: OptionalLong.empty position: 0 CreateTime: 1774875888100 size: 216 magic: 2 compresscodec: snappy crc: 2150642178 isvalid: true
| offset: 0 CreateTime: 1774875888100 keySize: 8 valueSize: 82 sequence: 0 headerKeys: [__TypeId__] key: PAY-O-33 payload: {"paymentId":"PAY-O-33","customerId":"CUST-2","t
otalAmount":99.99,"status":"PAID"}
baseOffset: 1 lastOffset: 1 count: 1 baseSequence: -1 lastSequence: -1 producerId: 0 producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: true isControl: true deleteH
orizonMs: OptionalLong.empty position: 216 CreateTime: 1774875888151 size: 78 magic: 2 compresscodec: none crc: 3247449494 isvalid: true
| offset: 1 CreateTime: 1774875888151 keySize: 4 valueSize: 6 sequence: -1 headerKeys: [] endTxnMarker: COMMIT coordinatorEpoch: 0
```